

Lesson 21 — Build the Model & Define Your First Function

Matches LEGO Teacher Portal lesson: "Music Machine".

Objective (I can...): I can build the functions model, define a named function, and call it to run a block of logic. **CS concept:** Procedural abstraction — defining and calling a function (procedure) · **Builds on:** Lesson 20 (programs are now big enough to need organizing).

AI Connection (2 min): Engineers package AI behavior into reusable functions — "detectFace()," "classify()" — and call them wherever needed instead of rewriting them. *First-principles question:* "If you had to run the same 10-step routine in five places, what's the risk of copying it five times?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **This is the unit's build lesson** — build the Functions model now and keep it assembled through Lesson 25.

60-minute flow - Warm-up (8): "Name the routine" — pairs take a repeated real-world routine (a free-throw routine, a pre-flight check) and give it a single name, then describe it by that name. That naming *is* abstraction — the core of a function. - **Concept + demo (10):** Show a program with the same 4-block routine pasted three times, then define it once as a function (e.g., "beepAndBlink") and call it three times; run both to show identical behavior with far less code. Name *define* vs. *call*. - **Build/Code (30):** Pairs **build the Functions model** (the unit's one build), then define one function that does a small routine and call it at least twice in their program. They log the function's name and job in the notebook. *Swap at 15 min.* - **Challenge (7):** Call the function from inside a loop and predict how many times the routine runs. - **Share + pack-up (5):** Pairs show the definition and the calls and explain why one definition beats three copies. **Leave the model built.**

Pair roles: Driver defines/codes, Navigator names the function and counts the calls. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a palette card and a worked define/call example to adapt. *Extension / fast-finisher* — define two functions. *Reteach* — write the routine once, confirm it works, then wrap it in a function definition together.

Exit check: Pair defines a named function and calls it two+ times, and can explain define vs. call. **At-home (optional):** Name three routines in your day; write each as a function name and list its steps once. ~10 min.
